

Analyse des problèmes v4 → v4.1

Fichier : analyse_problemes_v4_v4_1.md **Dossier** : wiki racine

Branche : master (wiki) **Auteur** : hprzeta · **MAJ** : 2026-06-10

Auteur : hprzeta

Date : 3 juin 2026

Branche : Riemann_Lab_C

Fichiers : compute_zeros_v4.py (référence) → compute_zeros_v4_1.py (cible)

Introduction

compute_zeros_v4.py (Phase C initiale) portait l'affinage Illinois en C/libmpfr (illinois_mpfr.c) mais souffrait de trois problèmes structurels : incohérence entre la fonction Z du code C et celle de Python, corruption de l'état GMP par partage du .so avant fork(), et biais de position intrinsèque à la troncature RS ordre 1.

compute_zeros_v4_1.py (commit 893f3b4) corrige ces trois problèmes, portant la vitesse de ~ 18 z/s à ~ 41 z/s sur $T=300$ ($\times 2.2$) et atteignant une précision de position $< 10^{-12}$ (vs 10^{-4} - 10^{-2} pour v4).

Problème 1 — Z_{double} interne dans `illinois_mpfr.c` : incohérence de signe

Cause mathématique

v4 faisait appel à `illinois_mpfr(a, b, tol)` qui réévaluait $Z(a)$ et $Z(b)$ côté C via Z_{mpfr} — la formule RS tronquée à l'ordre 1 ($C_0 + C_1$) :

$$Z_{\text{mpfr}}(t) = 2 \sum_{n=1}^{N(t)} \frac{\cos(\theta(t) - t \ln n)}{\sqrt{n}} + R_{C_0+C_1}(t)$$

Le reste $R_{C_0+C_1}$ diffère du vrai reste $R(t)$ d'une quantité $O(t^{-3/4})$. Pour $t < 300$ ($N < 7$ termes RS), cette approximation produit un **biais ~ 0.3** sur $Z_{\text{mpfr}}(t)$ — suffisant pour inverser un signe.

Conséquence : si Python détecte un bracket $[a, b]$ avec $Z_{\text{python}}(a) \cdot Z_{\text{python}}(b) < 0$, mais que $Z_{\text{mpfr}}(a)$ ou $Z_{\text{mpfr}}(b)$ a un signe différent de Z_{python} , Illinois C converge vers une **fausse racine** hors du bracket, ou diverge.

Solution : Option B — passer (f_a, f_b) depuis Python (commit 581e34d)

Supprimer l'évaluation de Z côté C pour l'initialisation. Les valeurs de bord sont calculées en Python (via `Z_vect_correct numpy, exact`) et passées à `illinois_refine` :

```
/* illinois_mpfr.c - Option B */
double illinois_refine(double a, double b,
                      double fa, double fb, /* Z(a), Z(b) calculés en Python */
                      int prec_bits, double tol, int max_iter);
```

Les itérations intermédiaires restent Z_{mpfr} C — correct pour $t \geq 300$ ($N \geq 7$ termes, précision $\sim 10^{-4}$) et inutile à affiner davantage car la convergence Illinois est quadratique dès que les bornes sont cohérentes.

Seuil de bascule : `T_SEUIL_ILLINOIS_C = 300.0` — $t < 300$ passe par `mpmath.findroot`.

Gain mesuré

Méthode affinage	Précision position γ	Biais observé
v4 <code>illinois_mpfr(a, b, tol)</code>	10^{-4} - 10^{-2}	jusqu'à 0.3 pour $t < 300$
v4.1 <code>illinois_refine(a, b, fa, fb, ...)</code>	$< 10^{-12}$ ✓	0 — bornes ancrées Python

Problème 2 — Chargement du .so avant `fork()` : sérialisation GMP

Cause

v4 chargeait `illinois_mpfr.so` dans le processus principal **avant** le multiprocessing. `Pool.map`. Après `fork()`, chaque worker héritait du **même handle ctypes** pointant vers la même instance GMP/MPFR en mémoire partagée (copy-on-write non garanti pour les structures internes GMP).

Conséquences mesurées (session 31 mai 2026) :

Scénario	Gain parallèle (W=4)
<code>Pool(sleep)</code> — aucun ctypes	$\times 3.9$
<code>Pool(worker_chunk)</code> v4 — .so pré-fork	$\times 1.84$

Gain parallèle réduit à $\times 1.84$ sur 4 workers théoriques — GMP sérialisait les appels.

Solution : chargement du .so post-fork (commit d9bb267)

```
def _worker_chunk(args):
    import ctypes as _ctypes
    _lib = _ctypes.CDLL(so_path) # chargé APRÈS fork - handle GMP isolé
    _lib.illinois_refine.restype = _ctypes.c_double
    _lib.illinois_refine.argtypes = [
        _ctypes.c_double, _ctypes.c_double, # a, b
```

```

        _ctypes.c_double, _ctypes.c_double, # fa, fb
        _ctypes.c_int, _ctypes.c_double, _ctypes.c_int,
    ]

```

Chaque processus fils instancie sa propre copie de libmpfr — zéro partage d'état GMP.

Gain mesuré

Scénario	Gain parallèle (W=4)
v4 pré-fork	×1.84
v4.1 post-fork	×3.9 ✓

Problème 3 — Bug Z_batch : $N_{\max}$ fixe pour tout le batch

Cause mathématique

La formule de Riemann-Siegel requiert $N(t) = \lfloor \sqrt{t/2\pi} \rfloor$ termes **propres à chaque** t . Z_batch dans v4 calculait :

```

N_max = int(np.floor(np.sqrt(t_max / (2 * np.pi)))) + 1 # FIXE pour tout le batch

```

Pour les $t_k < t_{\max}$ du batch, les termes $n \in (N(t_k), N_{\max}]$ étaient accumulés sans droit. L'erreur maximale :

$$\Delta Z(t_k) \leq 2 \sum_{n=N(t_k)+1}^{N_{\max}} \frac{1}{\sqrt{n}} \approx 4(\sqrt{N_{\max}} - \sqrt{N(t_k)})$$

À $t_k = 14$, batch $[14, 300]$: $\Delta N \approx 12$ termes excédentaires $\rightarrow \Delta Z \approx 3.5$, suffisant pour corrompre tous les changements de signe.

Solution : Z_vect_correct avec masque booléen (commit 50837f7)

```

mask = (np.arange(1, N_max + 1)[None, :] <= Ns[:, None]) # True ssi n <= N(t_k)
Z_out = 2.0 * np.dot(np.cos(phases) * mask, inv_sqn) # BLAS - 0 boucle Python

```

Gain mesuré

Méthode	Erreur max Z	Vitesse relative
Z_batch ($N_{\max}$ fixe)	jusqu'à $\sim 3.5 \times$	×4 000 vs scalaire
Z_vect_correct (masque)	$< 10^{-10}$ ✓	×9 000 vs scalaire

Tableau récapitulatif global

#	Problème	Correction v4.1	Gain me
1	Z_{double} incohérent — biais ~ 0.3 pour $t < 300$	Option B : (f_a, f_b) depuis Python	Précision
2	.so pré-fork — sérialisation GMP	Chargement post-fork par worker	Parallèle
3	N_{max} fixe z_{batch} — erreur RS jusqu'à 3.5	$z_{\text{vect_correct}}$ masque booléen	Erreur <

Résultats de validation

Vérif B — LMFDB T=10 000 (3 juin 2026)

Métrique	v4	v4.1
Vitesse	~ 18 z/s	41 z/s
Précision γ (LMFDB 20 premiers)	10^{-4} - 10^{-2}	19/20 $< 10^{-10}$ ✓
Biais $t < 300$	présent	absent ✓
Gain parallèle (W=4)	$\times 1.84$	$\times 3.9$
Turing-Backlund	COMPLET	COMPLET
Commit	—	581e34d

Questions ouvertes

1. **Vitesse affinage à grand t** : `mpmath.findroot` (fallback $t < 300$) coûte ~ 80 ms/zéro à $t \approx 500$. Pour T=10 000, les 138 zéros $t < 300$ prennent ~ 11 s (1.4 % du total, acceptable). Au-delà, `illinois_refine C` domine. Goulot restant : l'évaluation Z_{mpfr} dans `illinois_refine` (~ 85 ms/appel à grand t). **Résolu en v4.1+Arb par remplacement de Z_{mpfr} par Arb $\times 27$.**
2. **STEP adaptatif** : v4 et v4.1 initiales utilisaient STEP=0.1 fixe. La croissance de densité $\delta(t) = 2\pi / \ln(t/2\pi)$ impose STEP $< \delta/2$ pour ne pas rater de paires. **Résolu en v4.1 par `step_pour_t()` — STEP = $\delta(t)/3$, borné $[0.05; 0.5]$.**
3. **Segmentation workers** : v4 découpait en tranches uniformes de $t \rightarrow$ déséquilibre $\times 1.5$ du nombre de zéros. **Résolu en v4.1 par `_partitionner_adaptatif()` — segmentation par $\sqrt{t}$.**

analyse_problemes_v4_v4_1.md · wiki racine · master · hprzeta · MAJ 2026-06-10 · 189 lignes